

Structured Computer Architecture

Chapter 3.12: Von Neumann Architecture

Fourth-Order Systems (4-OS)

Based on Structured Computer Architecture: An Order-Based Approach

Outline

Learning Objectives

By the end of this chapter, you will understand:

- ▶ The historical context and theoretical foundations of computation
- ▶ The mathematical model of the Turing Machine
- ▶ The decision problem (Entscheidungsproblem) and its significance
- ▶ Von Neumann architecture principles and components
- ▶ The stored program concept and its implications
- ▶ Memory organization and addressing schemes
- ▶ Instruction set architecture fundamentals
- ▶ The relationship between hardware and software

The Turning Point: Fourth-Order Systems

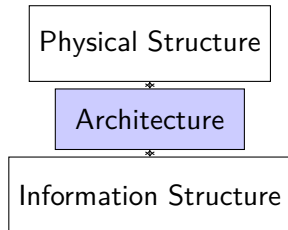
Fourth Loop Significance:

- ▶ Meeting point of physical and informational structures
- ▶ Circuit-information combination determines functionality
- ▶ Architecture as unifying concept
- ▶ Synthesis of symbolic and physical structures

Information as Control:

- ▶ Information takes control at this level
- ▶ Programs as symbolic structures
- ▶ Mathematical models enable computation
- ▶ Digital systems gain autonomy through loops

Architecture Concept



Origins of Computation Theory

The Epimenides Paradox (7th-6th

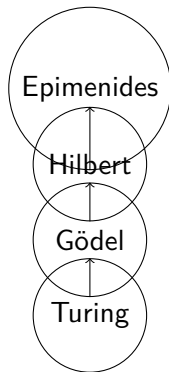
century BC):

- ▶ "All Cretans are liars" - spoken by a Cretan
- ▶ First recorded decision problem
- ▶ Cannot determine truth or falsehood
- ▶ Foundation of logical paradoxes

Historical Timeline:

- ▶ Ancient Greece: Logical paradoxes
- ▶ 1900: Hilbert's 23 problems
- ▶ 1928: Entscheidungsproblem formulated
- ▶ 1931: Gödel's incompleteness theorems
- ▶ 1936: Turing machine concept

Decision Problem Evolution



Hilbert's Entscheidungsproblem (1928)

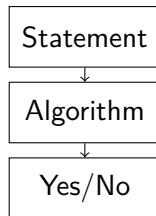
The Question:

"Is there an algorithm that takes a statement formulated in a formal system and answers with 'yes' or 'no' whether the statement is true or false in the given system?"

Significance:

- ▶ Fundamental question about computability
- ▶ Seeks universal decision procedure
- ▶ Bridges logic and computation
- ▶ Motivates formal system development

Decision Process



Gödel's Revolutionary Answer (1931)

Gödel's Incompleteness Theorem:

"In a formal system one can correctly construct true structures whose truth cannot be proven or disproved"

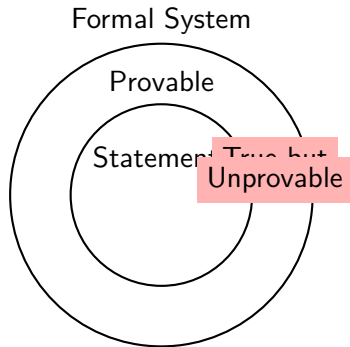
Implications:

- ▶ No universal decision algorithm exists
- ▶ Formal systems have inherent limitations
- ▶ Truth vs. provability distinction
- ▶ Foundation for computability theory

Impact on Computer Science:

- ▶ Triggered four independent works (1936)
- ▶ Church, Kleene, Post, Turing
- ▶ Founded theoretical computer science

Incompleteness Concept



Turing Machine Components

Three Essential Components:

1. **Infinite Tape:** Stores symbols from finite alphabet
2. **Read/Write Head:** Accesses tape, moves left/right
3. **Finite Automaton:** Controls head based on state and symbol

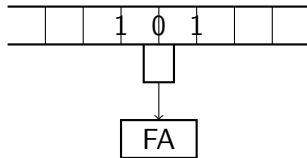
Operation Principle:

- ▶ Each clock cycle: read symbol, update state
- ▶ Head moves one position left or right
- ▶ Write new symbol based on current state
- ▶ Finite automaton determines all actions

Mathematical Abstraction:

- ▶ Infinite tape handles unknown space

Turing Machine Structure



Digital Implementation of Turing Machine

Digital Components:

- ▶ **Infinite RAM:** Replaces infinite tape
- ▶ **Up-Down Counter:** Addresses memory locations
- ▶ **Finite Automaton:** Same control logic

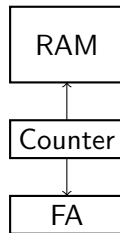
Practical Considerations:

- ▶ Finite memory in real implementations
- ▶ Counter provides sequential access
- ▶ Memory addressing enables random access
- ▶ Control logic remains unchanged

Theoretical vs. Practical:

- ▶ Mathematical infinity vs. finite resources
- ▶ Unknown space requirements handled by allocation

Digital Representation



The Stored Program Concept

Revolutionary Idea:

- ▶ Programs stored in same memory as data
- ▶ Instructions treated as data
- ▶ Dynamic program modification possible
- ▶ Universal computing machine

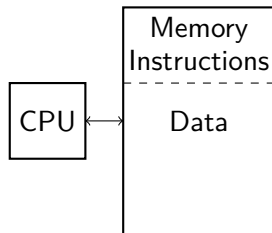
Key Principles:

- ▶ Single memory space for instructions and data
- ▶ Sequential instruction execution
- ▶ Binary representation of information
- ▶ Programmable control unit

Advantages:

- ▶ Flexibility in programming
- ▶ Self-modifying code capability

Stored Program Concept



Von Neumann Architecture Components

Essential Components:

1. **Central Processing Unit (CPU)**
2. **Memory Unit**
3. **Input/Output Devices**
4. **Control Unit**
5. **Arithmetic Logic Unit (ALU)**

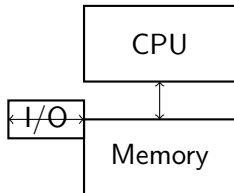
CPU Subcomponents:

- ▶ Control Unit: Instruction fetch and decode
- ▶ ALU: Arithmetic and logical operations
- ▶ Registers: Fast temporary storage
- ▶ Program Counter: Instruction address tracking

Data Flow:

- ▶ Instructions fetched from memory

Von Neumann Architecture



Memory Organization and Addressing

Memory Structure:

- ▶ Linear array of storage locations
- ▶ Each location has unique address
- ▶ Fixed word size (typically 8, 16, 32, 64 bits)
- ▶ Random access capability

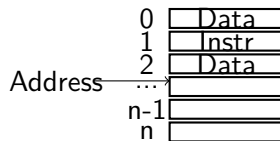
Addressing Schemes:

- ▶ Direct addressing: Address specifies location
- ▶ Indirect addressing: Address points to address
- ▶ Indexed addressing: Base + offset calculation
- ▶ Relative addressing: PC + displacement

Memory Hierarchy:

- ▶ Registers: Fastest, smallest capacity
- ▶ Cache: Fast, medium capacity

Memory Organization



Instruction Format and Types

Instruction Components:

- ▶ **Opcode:** Operation to be performed
- ▶ **Operands:** Data or addresses for operation
- ▶ **Addressing Mode:** How operands are specified

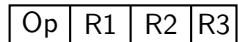
Instruction Types:

- ▶ Data transfer: Load, store, move
- ▶ Arithmetic: Add, subtract, multiply, divide
- ▶ Logical: AND, OR, NOT, XOR
- ▶ Control: Branch, jump, call, return
- ▶ I/O: Input, output operations

Instruction Formats:

- ▶ Fixed length: All instructions same size
- ▶ Variable length: Instructions of varying sizes

Instruction Format



6 bits 5 bits 5 bits 5 bits

ADD R1, R2, R3
 $R1 \leftarrow R2 + R3$

Fetch-Decode-Execute Cycle

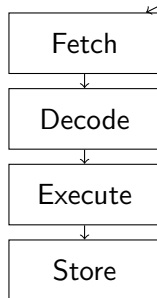
The Basic Cycle:

1. **Fetch:** Get instruction from memory
2. **Decode:** Interpret instruction format
3. **Execute:** Perform the operation
4. **Store:** Write results back

Detailed Steps:

- ▶ PC contains address of next instruction
- ▶ Instruction fetched from memory[PC]
- ▶ PC incremented to next instruction
- ▶ Control unit decodes instruction
- ▶ Operands fetched if needed
- ▶ ALU performs operation
- ▶ Results stored in destination

Fetch-Decode-Execute



The Architecture Abstraction

Abstraction Layers:

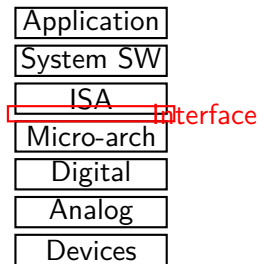
- ▶ Application software
- ▶ System software (OS, compilers)
- ▶ Instruction Set Architecture (ISA)
- ▶ Microarchitecture
- ▶ Digital logic circuits
- ▶ Analog circuits and devices

ISA as Interface:

- ▶ Contract between hardware and software
- ▶ Defines visible processor state
- ▶ Specifies instruction semantics
- ▶ Enables software portability

Implementation Independence:

Abstraction Layers



Performance Considerations

Performance Metrics:

- ▶ Execution time (latency)
- ▶ Throughput (bandwidth)
- ▶ Instructions per second (IPS)
- ▶ Clock frequency

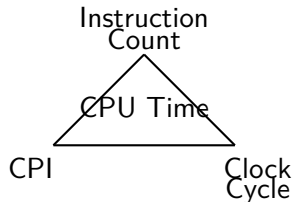
Performance Factors:

- ▶ Instruction count
- ▶ Clock cycles per instruction (CPI)
- ▶ Clock cycle time
- ▶ Memory access time

Performance Equation:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time} \quad (1)$$

Performance Triangle



Chapter Summary

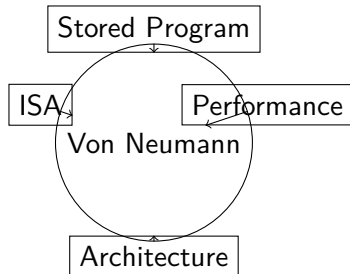
Key Concepts Covered:

- ▶ Fourth-order systems mark the convergence of physical and informational structures
- ▶ The decision problem led to fundamental computability theory
- ▶ Turing machines provide universal computation model
- ▶ Von Neumann architecture enables stored program concept
- ▶ ISA defines hardware-software interface
- ▶ Performance depends on instruction count, CPI, and clock cycle time

Historical Significance:

- ▶ From ancient paradoxes to modern computers
- ▶ Mathematical foundations enable practical

Von Neumann Legacy



Discussion Questions

1. How does the stored program concept enable universal computation?
2. What are the trade-offs between different instruction set architectures?
3. How do abstraction layers facilitate computer system design?
4. What role does the ISA play in software portability?
5. How can performance be optimized at the architecture level?
6. What are the implications of Gödel's incompleteness theorem for computing?
7. How does the Von Neumann bottleneck affect system performance?
8. What alternatives to Von Neumann architecture exist?